Raytrace benchmark

1. Benchmark Analysis:

The benchmark simulates a raytracer that generates a 3D scene onto a 2D canvas (100x100 pixels). The scene includes multiple objects (spheres and a plane) and light sources. The benchmark uses pure Python and basic OOP (classes for Vector, Point, Ray, Sphere, and surfaces). It uses standard modules like math for calculations, array for pixel buffer storage, and pyperf for timing.

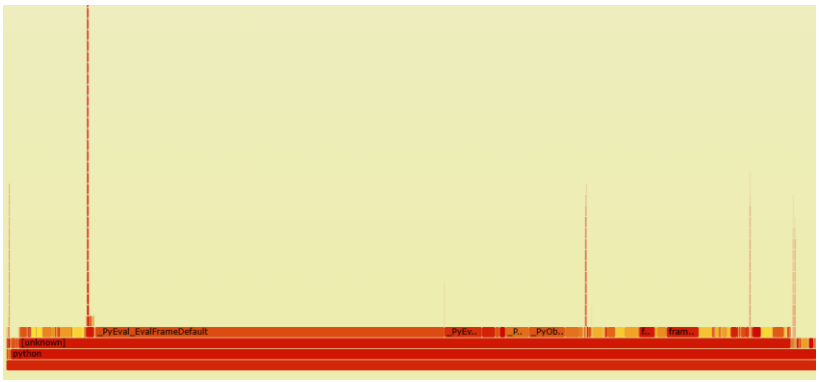
The algorithm goes through every pixel, shoots rays from the camera, and calculates intersections with objects to compute color, shadows, and reflections. This requires heavy 3D math (dot products, cross products, additions, scaling, and square roots) for each ray and pixel. The Vector and Point classes carry three scalar floating-point attributes (x, y, z) to perform standard 3D arithmetic. Because these math operations run inside nested loops and constantly create new Vector/Point objects, we expect these loops and the Python object overhead to be the main bottleneck.

2. Detecting Bottlenecks:

```
raytrace: Mean +- std dev: 799 ms +- 6 ms
```

```
Performance counter stats for 'python3 -m pyperformance run --bench raytrace':

      71887.54 msec task-clock                #    0.997 CPUs utilized
          2327      context-switches          #   32.370 /sec
             0      cpu-migrations            #    0.000 /sec
        122262      page-faults               #    1.701 K/sec
  170641168313      cycles                    #    2.374 GHz
  437988756034      instructions              #    2.57  insn per cycle
   84465677836      branches                  #    1.175 G/sec
     333920604      branch-misses             #    0.40% of all branches

    72.108435500 seconds time elapsed

    71.441275000 seconds user
     0.496097000 seconds sys
```

```
49.36%     0.00%  python       [unknown]                          [.] 0000000000000000
       |
       ---0
          |
          |--42.25%--_PyEval_EvalFrameDefault
          |
          |--1.01%--float_dealloc.lto_priv.0
          |
          |--0.92%--PyObject_SetAttr
          |
          |--0.75%--_PyFunction_Vectorcall
          |
           --0.54%--_PyEval_MakeFrameVector

43.10%    43.09%  python       python3.10                         [.] _PyEval_EvalFrameDefault
       |
        --42.24%--0
                  _PyEval_EvalFrameDefault

 4.68%     4.68%  python       python3.10                         [.] _PyEval_MakeFrameVector
       |
        --0.53%--0
                  _PyEval_MakeFrameVector

 4.44%     4.44%  python       python3.10                         [.] _PyObject_GetMethod
 4.22%     4.21%  python       python3.10                         [.] frame_dealloc.lto_priv.0
 2.90%     2.90%  python       python3.10                         [.] _PyObject_GenericSetAttrWithDict
 1.92%     1.92%  python       python3.10                         [.] float_mul.lto_priv.0
 1.74%     1.74%  python       python3.10                         [.] _PyObject_MakeTpCall
 1.66%     1.66%  python       python3.10                         [.] float_dealloc.lto_priv.0
       |
        --1.01%--0
                  float_dealloc.lto_priv.0
```



Looking at the perf stats, perf report, and Flame Graph, the primary bottlenecks in the baseline are:

- Python Interpreter Overhead:

The main interpreter evaluation loop (_PyEval_EvalFrameDefault) consumes 43.10% of the runtime. The Flame Graph shows a very wide base, meaning almost half of the runtime is spent

on evaluating bytecode and running loop logic inside Python.

- Object Creation and Attribute Handling:
The perf report reveals that managing classes and calling methods takes a large portion of execution time:

- _PyObject_GetMethod (4.44%): Looking up methods on objects during raytracing steps.
- _PyObject_GenericSetAttrWithDict (2.90%): Writing object attributes into Python's dynamic storage.
- _PyObject_MakeTpCall (1.74%): Overhead from instantiating new Python objects.

Because the code creates separate Vector, Point, and Ray objects for every single pixel and light check, Python wastes significant time constantly allocating and looking up object properties.

- Function Call and Frame Overhead:
_PyEval_MakeFrameVector (4.68%) and frame_dealloc (4.22%): Together, creating and destroying function call frames takes almost 9% of total runtime. Breaking simple vector math into many tiny helper methods forces Python to constantly build and clean up interpreter call stacks.

- Floating-Point and Math Overhead:
Math operations like float_mul (1.92%) and memory management like float_dealloc (1.66%) show repeated overhead from constantly creating and destroying temporary float numbers during 3D calculations.

3. Optimization:

We chose to make the following changes:
-Adding slots to classes: We used slots on classes like Vector, Point, Ray, and Sphere so Python avoids creating heavy internal dictionaries for each object, making object creation and attribute lookups much faster.

- Pre-calculating constant math: In the original implementation, the constant expression self.radius * self.radius was re-evaluated inside intersectionTime() during every intersection check. In a 100x100 frame there are over 100,000 sphere tests, so we calculated self.radius_sq once in init function, and saved the constant result to avoid many repeated floating-point calculations.

- Writing math calculations directly: Instead of calling helper methods and creating temporary Vector objects for basic steps, we wrote the coordinate math (x, y, z) directly inside methods like Vector.normalized(), Vector.reflectThrough(), and Halfspace.intersectionTime().

The Code:
Before:

```
# BEFORE: Unoptimized Vector
class Vector(object):
    def __init__(self, initx, inity, initz):
        self.x = initx
        self.y = inity
        self.z = initz

    def dot(self, other):
        other.mustBeVector() # ❌ Overhead: Runtime type check on every dot product
        return (self.x * other.x) + (self.y * other.y) + (self.z * other.z)

    def normalized(self):
        return self.scale(1.0 / self.magnitude()) # ❌ Multiple nested method calls
```

```
# BEFORE: Unoptimized Sphere
class Sphere(object):
    def __init__(self, centre, radius):
        centre.mustBePoint()
        self.centre = centre
        self.radius = radius

    def intersectionTime(self, ray):
        cp = self.centre - ray.point
        v = cp.dot(ray.vector)
        # ❌ Re-calculating radius * radius for every ray test
        discriminant = (self.radius * self.radius) - (cp.dot(cp) - v * v)
        if discriminant < 0:
            return None
        return v - math.sqrt(discriminant)
```

After:

```
# AFTER: Optimized Vector
class Vector(object):
    __slots__ = ('x', 'y', 'z') # Prevents dict creation, reduces memory & speeds up lookup

    def __init__(self, initx, inity, initz):
        self.x = initx
        self.y = inity
        self.z = initz

    def dot(self, other):
        # ⚡ Direct multiplication without type-checking methods
        return (self.x * other.x) + (self.y * other.y) + (self.z * other.z)

    def normalized(self):
        # ⚡ Inlined magnitude computation to avoid redundant calls
        inv_mag = 1.0 / math.sqrt(self.x * self.x + self.y * self.y + self.z * self.z)
        return Vector(self.x * inv_mag, self.y * inv_mag, self.z * inv_mag)
```

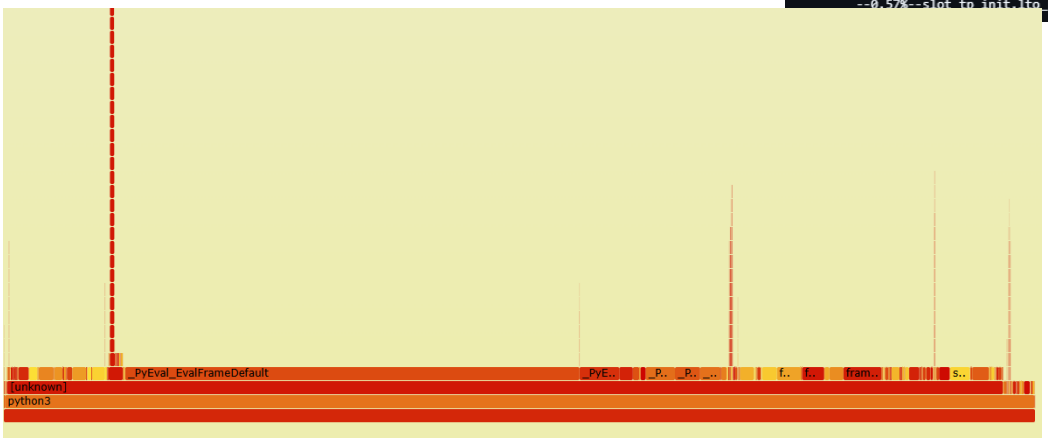
```python
# AFTER: Optimized Sphere
class Sphere(object):
    __slots__ = ('centre', 'radius', 'radius_sq')

    def __init__(self, centre, radius):
        self.centre = centre
        self.radius = radius
        self.radius_sq = radius * radius # 🚀 Pre-computed once at object creation

    def intersectionTime(self, ray):
        cp = self.centre - ray.point
        v = cp.dot(ray.vector)
        # ⚡ Reusing pre-computed radius_sq inside the trace loop
        discriminant = self.radius_sq - (cp.dot(cp) - v * v)
        if discriminant < 0:
            return None
        return v - math.sqrt(discriminant)
```

These modifications resulted in:

```
raytrace_optimized: Mean +- std dev: 449 ms +- 4 ms
```

```
Performance counter stats for 'python3 raytrace_benchmark_optimized.py':

      40470.70 msec task-clock                #    0.996 CPUs utilized
          1570      context-switches          #   38.793 /sec
             0      cpu-migrations            #    0.000 /sec
         79322      page-faults               #    1.960 K/sec
     96137459555      cycles                    #    2.375 GHz
    247623259273      instructions              #    2.58  insn per cycle
     46410198688      branches                  #    1.147 G/sec
       220535388      branch-misses             #    0.48% of all branches

    40.644606769 seconds time elapsed

    40.174394000 seconds user
     0.329914000 seconds sys
```

```
51.94%     0.00%  python3  [unknown]                          [k] 0000000000000000
       |
       ---0
          |
          |--43.38%--_PyEval_EvalFrameDefault
          |
          |--1.40%--float_dealloc.lto_priv.0
          |
          |--0.88%--PyObject_SetAttr
          |
          |--0.79%--listiter_next.lto_priv.0
          |
           --0.64%--_PyFunction_Vectorcall

44.06%    44.04%  python3  python3.10                        [.] _PyEval_EvalFrameDefault
       |
        --43.36%--0
                  _PyEval_EvalFrameDefault

 4.00%     4.00%  python3  python3.10                        [.] frame_dealloc.lto_priv.0
 3.93%     3.93%  python3  python3.10                        [.] _PyEval_MakeFrameVector
 2.84%     2.84%  python3  python3.10                        [.] _PyObject_GenericSetAttrWithDict
 2.44%     2.44%  python3  python3.10                        [.] _PyObject_GetMethod
 2.33%     2.33%  python3  python3.10                        [.] float_dealloc.lto_priv.0
       |
        --1.40%--0
                  float_dealloc.lto_priv.0

 2.15%     2.15%  python3  python3.10                        [.] float_mul.lto_priv.0
 1.97%     1.97%  python3  python3.10                        [.] _PyObject_MakeTpCall
 1.92%     1.92%  python3  python3.10                        [.] subtype_dealloc.lto_priv.0
 1.64%     1.64%  python3  python3.10                        [.] tupledealloc.lto_priv.0
 1.61%     1.61%  python3  python3.10                        [.] slot_tp_init.lto_priv.0
       |
        --0.57%--slot_tp_init.lto_priv.0
```



- Mean Execution Time: Dropped from 799 ms to 449 ms (~43.8% runtime reduction, 1.78x speedup).

- Instructions: Reduced by 43.46% (from 437.99B to 247.62B), eliminating billions of internal interpreter evaluation steps.

- CPU Cycles: Dropped by 43.66% (from 170.64B to 96.14B), directly driving the speedup.

Page Faults: Decreased by 35.12% (from 122,262 to 79,322), reflecting a smaller memory footprint.

Stable IPC: Remained steady at 2.58 (baseline: 2.57).

Remaining Bottlenecks and Trade-offs:

- Interpreter Overhead (_PyEval_EvalFrameDefault): Still consumes 43.38% of the optimized runtime (baseline: 42.25%), because the code remains in pure Python without native compilation.

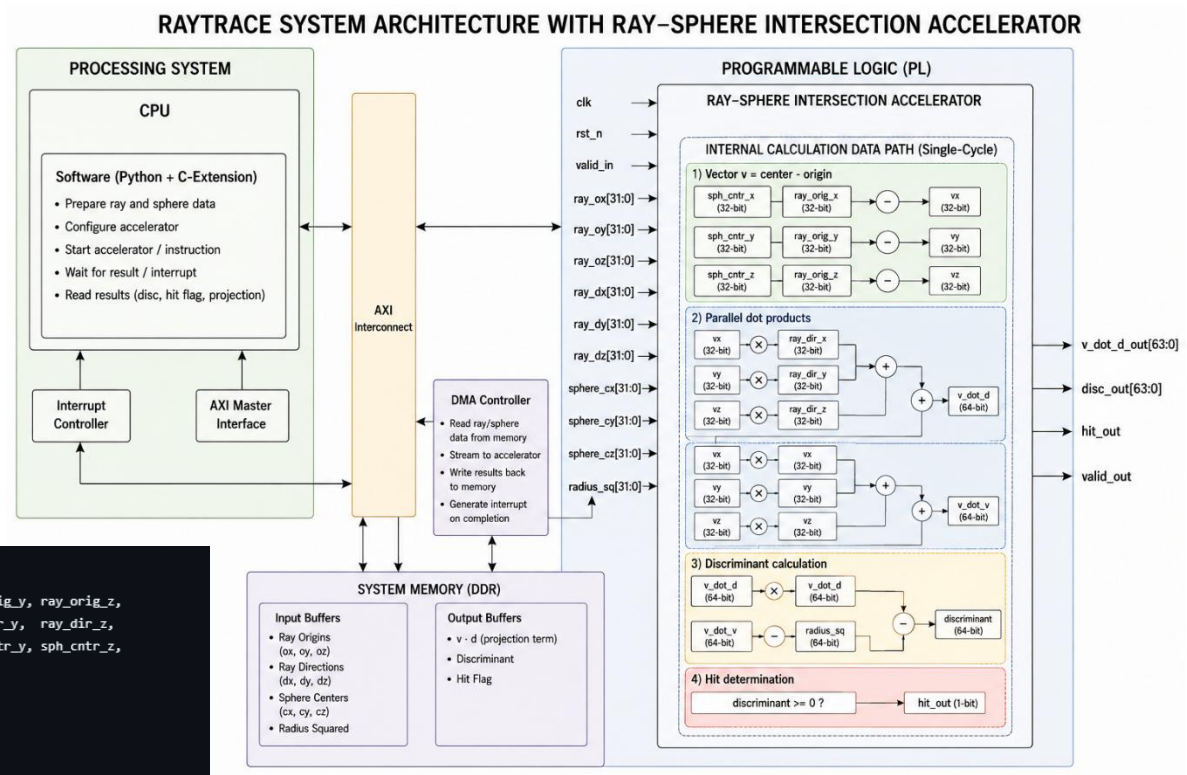
- Branch Mispredictions: The miss rate slightly increased from 0.40% to 0.48%. The miss rate is calculated as `Mispredicted Branches / Total Branches`. The optimized code removed a lot of simple, predictable code, which significantly lowered the total branch count (the denominator). Because the denominator shrank while the small number of tricky branches stayed the same, the resulting percentage went up slightly.

- Context Switches: The rate slightly increased from 32.370 /sec to 38.793 /sec. The total count actually dropped from 2,327 down to 1,570, but dividing by a much shorter total runtime (40.6s vs 72.1s) makes the per- second rate look higher.

4. Hardware acceleration:

The biggest bottleneck in Raytrace is calculating ray-sphere intersections for every ray and pixel. In Python, this is split into many separate steps: subtracting 3D vectors, calculating multiple dot products, squaring values, and evaluating the discriminant formula. This creates a lot of instruction overhead on the CPU. We want a single hardware block that does this entire calculation at once in parallel.

We implemented a hardware unit in SystemVerilog, which takes the ray and sphere parameters and checks the intersection in a single clock cycle.




```systemverilog
// Ray-Sphere Intersection Acceleration Unit
module ray_sphere_intersector (
    input  logic signed [31:0] ray_orig_x, ray_orig_y, ray_orig_z,
    input  logic signed [31:0] ray_dir_x,  ray_dir_y,  ray_dir_z,
    input  logic signed [31:0] sph_cntr_x, sph_cntr_y, sph_cntr_z,
    input  logic signed [31:0] radius_sq,

    output logic signed [63:0] v_dot_d_out,
    output logic signed [63:0] disc_out,
    output logic               hit_out
);
    // Vector v = center - origin
    logic signed [31:0] vx, vy, vz;
    assign vx = sph_cntr_x - ray_orig_x;
    assign vy = sph_cntr_y - ray_orig_y;
    assign vz = sph_cntr_z - ray_orig_z;

    // Parallel dot products
    logic signed [63:0] v_dot_d, v_dot_v;
    assign v_dot_d = (vx * ray_dir_x) + (vy * ray_dir_y) + (vz * ray_dir_z);
    assign v_dot_v = (vx * vx) + (vy * vy) + (vz * vz);

    // Discriminant formula
    logic signed [63:0] discriminant;
    assign discriminant = (v_dot_d * v_dot_d) - (v_dot_v - radius_sq);

    // Outputs
    assign v_dot_d_out = v_dot_d;
    assign disc_out    = discriminant;
    assign hit_out     = (discriminant >= 0);
endmodule
```

Inputs and Outputs:

- clk, rst_n: System clock and active-low reset.
- ray_ox, ray_oy, ray_oz: 32-bit signed fixed-point coordinates of the ray origin.
- ray_dx, ray_dy, ray_dz: 32-bit signed fixed-point coordinates of the ray direction.
- sphere_cx, sphere_cy, sphere_cz: 32-bit signed fixed-point coordinates of the sphere center.
- radius_sq: 64-bit unsigned fixed-point pre-computed squared radius.
- v_dot_d_out: 64-bit signed projection term (v · d).
- disc_out: 64-bit signed discriminant result.
- hit_out: 1-bit boolean flag indicating if the ray intersects the sphere (discriminant >= 0).
- valid_in / valid_out: 1-bit control signals for data readiness.

Hardware / Software Interface & Integration:

- The unit is designed to work as an instruction extension directly inside the CPU pipeline, or connected via a standard memory-mapped bus (like AXI).

- Instead of the CPU executing multiple vector math operations and branches, the software triggers this single hardware instruction, passes ray/sphere vectors via registers, and receives the discriminant and hit flag immediately.

- Software Modifications: The Python `Sphere.intersectionTime` method offloads calculation via a C-extension wrapper that passes vector values directly to the hardware unit, bypassing temporary `Vector` object allocations and interpreter overhead.

The expected improvements:

- Performance: Replaces around 15-20 separate CPU instructions with 1 single cycle.

- Area: Small design using a few parallel multipliers and adders.

- Power: Saves power by avoiding repetitive CPU instruction decoding and temporary object allocations.

5. Conclusions:

Profiling the Raytrace benchmark showed that object allocation, dynamic attribute lookups, and repeated 3D math inside deep pixel loops were the primary bottlenecks.

Applying pure Python optimizations, using slots, pre-computing invariant values, and inlining vector math, reduced instructions and CPU cycles by ~43%, achieving a 1.78x speedup.

However, because the code remained in pure Python without native compilation, the interpreter evaluation loop (_PyEval_EvalFrameDefault) still dominated the execution time (~43%).

Finally, our proposed SystemVerilog hardware unit targets the core ray-sphere intersection bottleneck by computing vector math and the discriminant in a single clock cycle, replacing multiple CPU instructions with minimal hardware area.